Transformer-Based Electricity Consumption Forecasting

Multivariate time-series forecasting of household electricity demand

William Peng · Ciara Cameron · Christopher Pedretti

MSML612 — Deep Learning | Final Project | August 2026

Introduction — The Problem

Why household-level load forecasting is hard

- Electricity cannot be stored cheaply at grid scale — supply must match demand continuously.
- Short-term load forecasting drives generation scheduling, demand-response, and home battery dispatch.
- Aggregate demand is smooth: thousands of appliance events average out.
- A single household is spiky and heavy-tailed — a few discrete, high-draw events dominate.
- The variance that matters operationally lives in the peaks, not the daily rhythm.

Our question

Does self-attention capture enough structure in single-household demand to beat the forecasts available for free?

2.08M

raw minute-level readings

4 yrs

of continuous metering

Data Preparation

UCI Individual Household Electric Power Consumption

1 Clean & index

Combine date/time into a timestamp index, coerce to numeric, drop duplicate timestamps from DST transitions.

2 Resample hourly

Minute resolution is noisier than the task needs. Hourly means absorb isolated gaps: 2.08M rows → 34,589 hours.

3 Interpolate

~1.25% of raw rows have missing values. Time-weighted interpolation fills remaining hours without fragmenting the series.

4 Calendar features

Hour, day, month as sine/cosine pairs plus weekend flag. Keeps hour 23 adjacent to hour 0. 7 measured + 7 derived = 14.

5 Split, then scale

70/15/15 chronological. StandardScaler fit on train only — fitting before splitting leaks test statistics.

Why chronological splitting matters

Neighbouring hours are highly correlated. A shuffled test set would mostly contain hours whose immediate neighbours the model has already seen — reported performance would measure interpolation, not forecasting.

Proposed Solution

An encoder-only Transformer, and an evaluation designed to be falsifiable

Attention over recurrence

In an LSTM, hour 1 of the window reaches the prediction through 23 sequential state updates. Self-attention connects any two positions in one operation — constant dependency length regardless of separation.

Built for the daily cycle

The most informative context is often the same hour on previous days, not the previous hour. Attention represents that directly as a high weight on a distant position.

Pre-norm encoder layers

LayerNorm before each sublayer rather than after: better-conditioned gradients at initialisation, no warmup schedule required.

Falsifiable evaluation

We report skill against naive persistence, not metrics in isolation. If the model cannot beat repeating the last hour, it is not worth its complexity.

Tools and Technologies

PyTorch 2.x

nn.TransformerEncoder, custom positional encoding, Adam, ReduceLROnPlateau

scikit-learn

StandardScaler fit on training split, MAE / MSE metrics

pandas + NumPy

Resampling, time-weighted interpolation, cyclical feature engineering

Matplotlib

All figures generated from one shared styling module

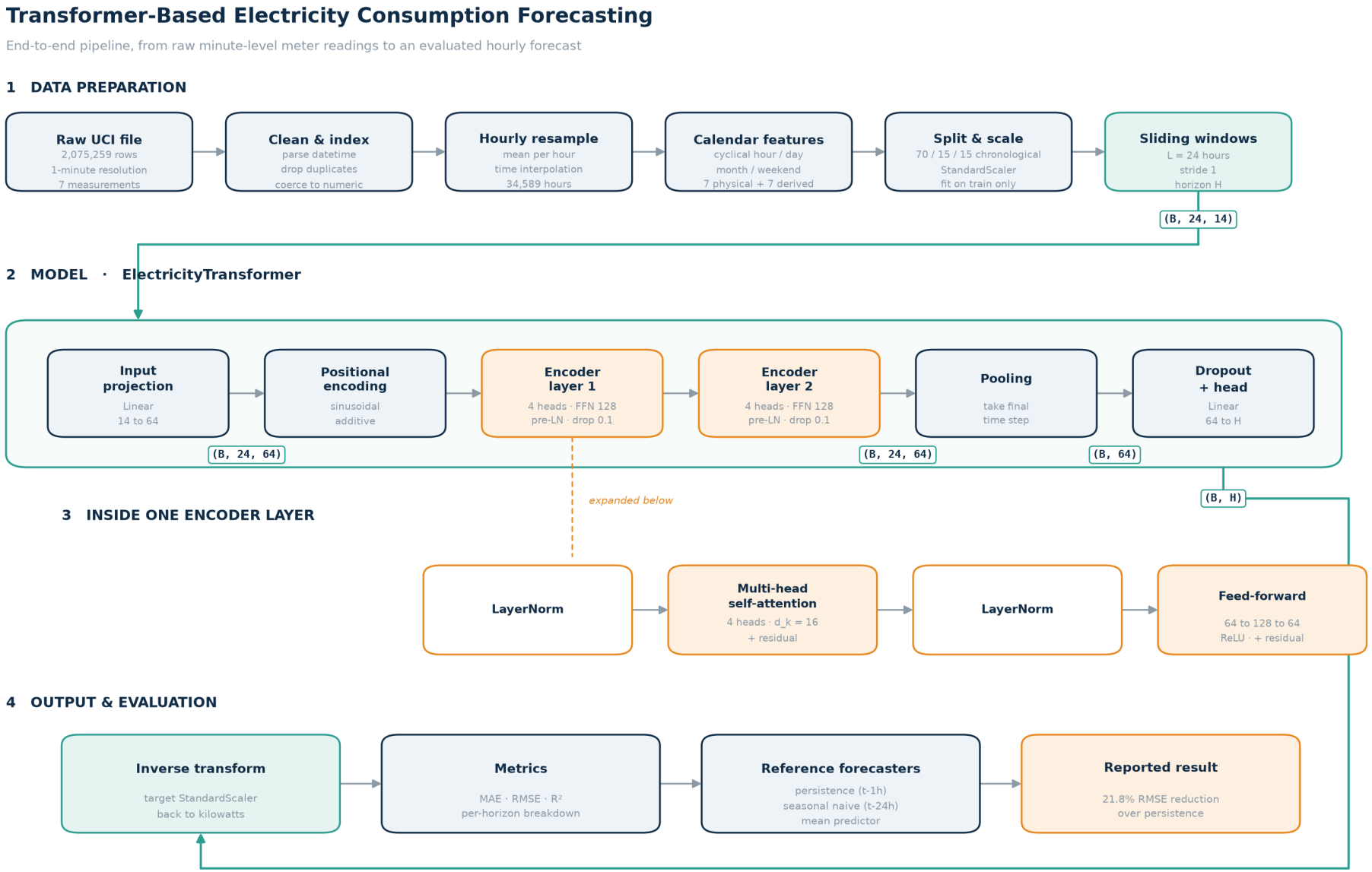
Streamlit

Live inference demo running on CPU

Git + GitHub

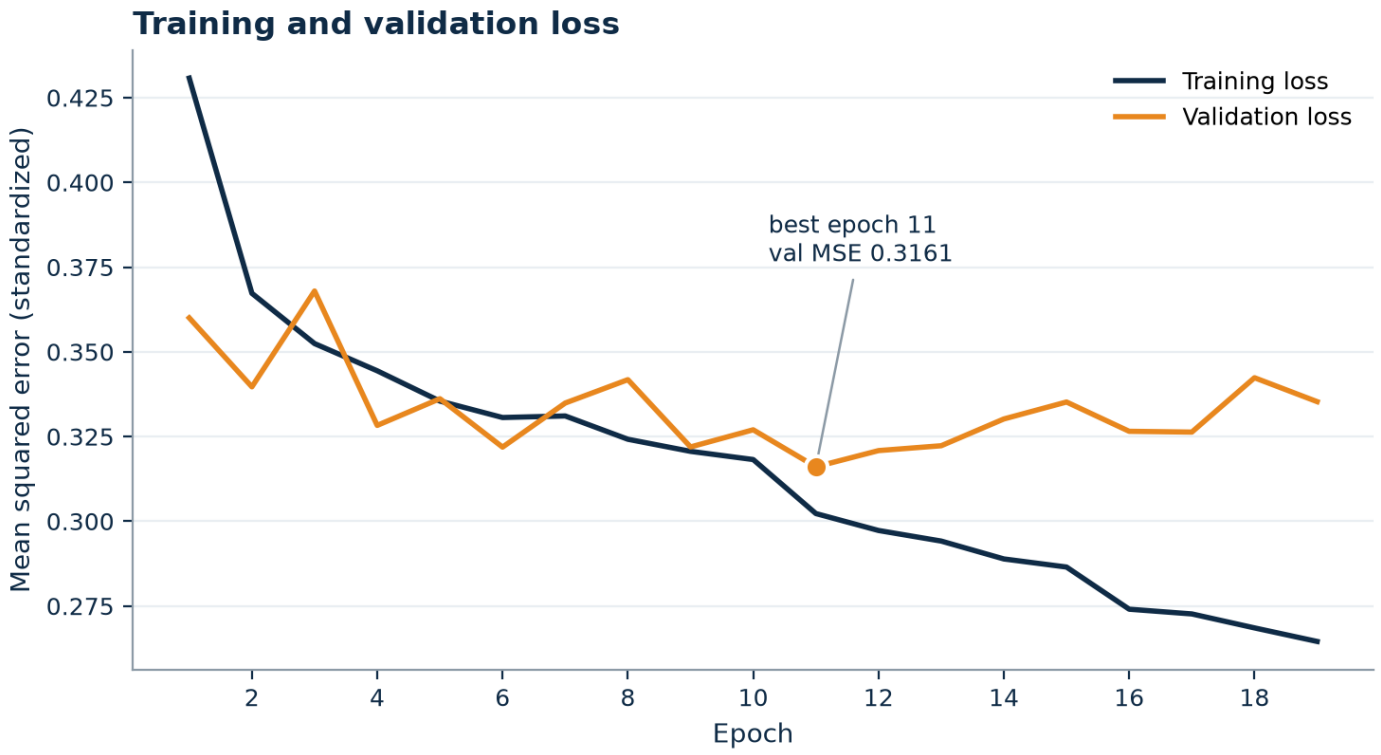
Public repo, pinned requirements, per-run history JSON

Architecture Diagram



Training

Required plot: training and validation loss vs. epoch



What this curve told us

Our interim run overfit from epoch 9 to 20 — validation rising while training fell. The final run stops that: best validation 0.3161 at epoch 11, LR halved as improvement stalls, training halted at epoch 19.

- Adam, learning rate 1e-3, MSE on standardized targets
- Gradient-norm clipping at 1.0
- ReduceLROnPlateau — halve LR after 3 stalled epochs
- Early stopping, patience 8; best checkpoint retained
- All seeds fixed at 42; every run writes a history JSON

Evaluation Methodology

A deep model is only interesting if it beats the forecast you get for free

Naive persistence

Predict the previous hour

The standard benchmark for hourly load — and a strong one

Seasonal naive

Predict the same hour yesterday

Tests whether the daily cycle alone is predictive

Mean predictor

Always predict the series mean

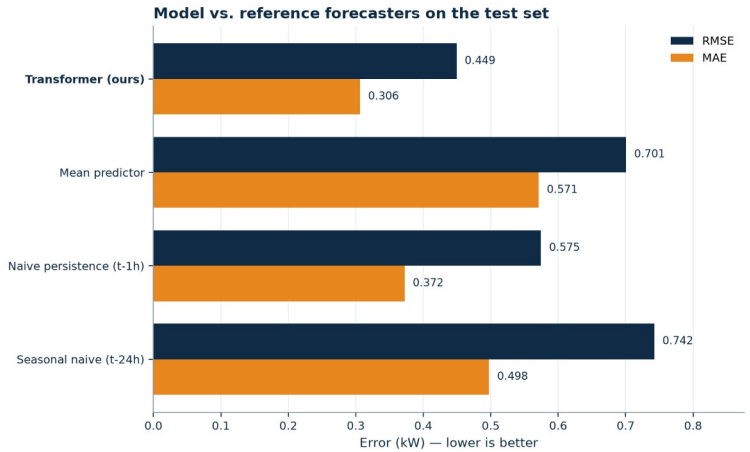
The $R^2 = 0$ reference point

Metrics: MAE, RMSE, and R^2, all computed in kilowatts after inverse-transforming predictions. MAE and RMSE together are informative — their ratio shows how much error is concentrated in a few large misses. Headline measure: skill score, the percentage RMSE reduction against persistence.

Results

5,165 held-out hourly predictions the model never saw in training

Model	MAE (kW)	RMSE (kW)	R^2
Transformer (ours)	**0.3062**	**0.4494**	**0.589**
Naive persistence (t−1h)	0.3724	0.5745	0.327
Mean predictor	0.5711	0.7005	0.000
Seasonal naive (t−24h)	0.4976	0.7424	−0.121



21.8%

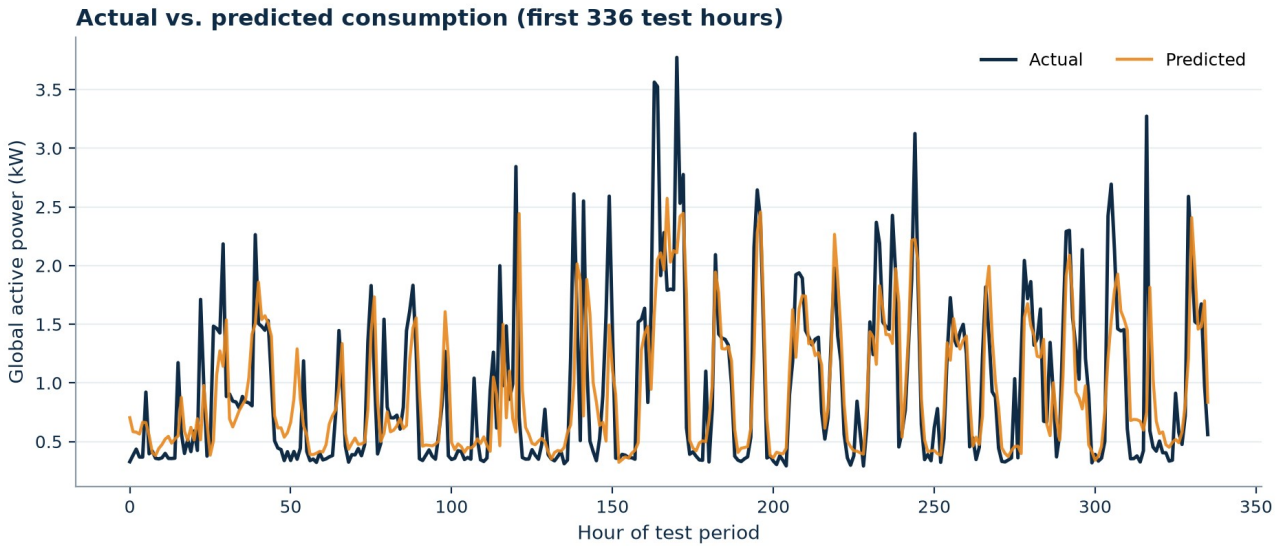
RMSE reduction vs. naive persistence

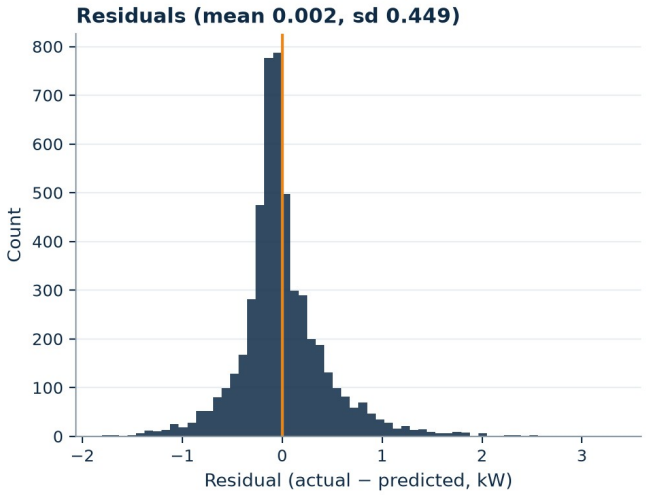
Two things worth noticing

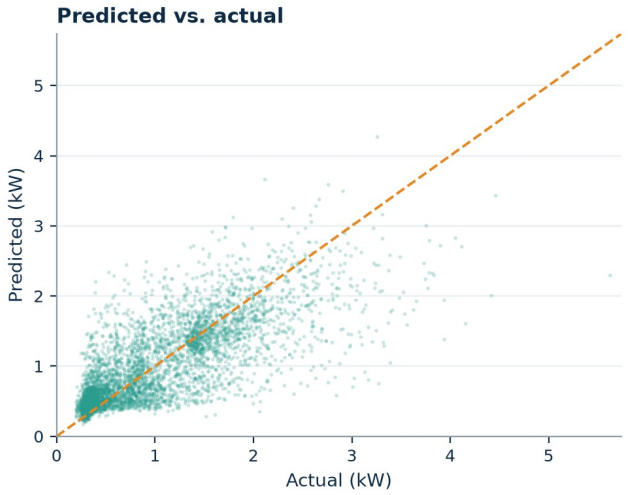
Seasonal naive scores worse than the mean predictor — a negative R^2. Household routines vary too much day to day for yesterday's 7pm to predict today's.

Our margin is larger in RMSE (21.8%) than MAE (17.8%): the gain is concentrated exactly where persistence fails worst.

Where the Model Still Struggles







Peak under-prediction

The model tracks the daily cycle and event timing, but consistently under-shoots peak magnitude — across the test period predicted maxima reach 4.27 kW against observed 5.63 kW.

Residuals are right-skewed; the scatter falls below the diagonal at high load.

This is rational under MSE. A confident spike that lands an hour off is punished twice — missed peak plus false peak — so hedging toward the mean is optimal wherever the data is most volatile.

It is a loss-function artifact, not a capacity failure.

Live Demo

Forecasting from any point in the held-out test period

```
streamlit run demo/app.py
```

- Pick a moment in the test period — the model forecasts the next hour from the previous 24
- Prediction, ground truth, and absolute error against the surrounding load curve
- Performance tab: live baseline comparison · About tab: architecture and training curve

Ablations

Every row is an independent train-and-evaluate cycle

Study	Final choice	RMSE	Alternatives	RMSE
Window	**24 hours**	**0.4494**	48 h / 168 h	0.4543 / 0.4553
Capacity	**d=64, 2 layers**	**0.4494**	2× depth / 2× width	0.4539 / 0.4560
Positional	**Sinusoidal**	**0.4494**	Learnable / none	0.4573 / 0.4542
Features	**With calendar**	**0.4494**	Without calendar	0.4617
Architecture	**Transformer**	**0.4494**	LSTM control	0.4579

What the grid says

Calendar features are the biggest single factor — dropping them costs more than any architecture change.

More capacity only hurts: the constraint is data, not expressiveness.

Transformer beats the LSTM control, 21.8% vs. 20.3% skill.

Every config still beats persistence by ≥ 19.6% — the conclusion is robust.

Limitations and Future Scope

Limitations

- Single household in a single location — no claim about aggregate or cross-household performance
- No exogenous variables; temperature is the strongest known driver of residential demand and is absent
- Point forecasts with no uncertainty bounds — the wrong output format for the peak problem
- One-hour horizon; operational scheduling needs 24 hours ahead
- Single seed — small differences between ablation rows should not be over-read

Future work

- Quantile / pinball loss — let the model express uncertainty about peaks instead of hedging
- Join hourly weather for Sceaux over the collection period
- Multi-step forecasting to 24 hours; the code already supports the horizon parameter
- Visualise attention weights to test whether the model really attends to the same hour on prior days
- Evaluate on a multi-household dataset to test transfer

References

[1] Vaswani et al. Attention is all you need. NeurIPS, 2017.

[2] Hochreiter & Schmidhuber. Long short-term memory. Neural Computation, 1997.

[3] Zhou et al. Informer: Beyond efficient transformer for long sequence time-series forecasting. AAAI, 2021.

[4] Wu et al. Autoformer: Decomposition transformers with auto-correlation. NeurIPS, 2021.

[5] Zeng et al. Are transformers effective for time series forecasting? AAAI, 2023.

[6] Lim et al. Temporal fusion transformers for interpretable multi-horizon forecasting. IJF, 2021.

[7] Kong et al. Short-term residential load forecasting based on LSTM RNN. IEEE Trans. Smart Grid, 2019.

[8] Hyndman & Athanasopoulos. Forecasting: Principles and Practice, 3rd ed. OTexts, 2021.

[9] Hébrail & Bérard. Individual Household Electric Power Consumption. UCI ML Repository, 2006.

[10] Kingma & Ba. Adam: A method for stochastic optimization. ICLR, 2015.

[11] Xiong et al. On layer normalization in the transformer architecture. ICML, 2020.

[12] Nie et al. A time series is worth 64 words. ICLR, 2023.

Questions

github.com/Ciaracam/612-Group-Project

William Peng · Ciara Cameron · Christopher Pedretti